

Antworten zu den Projektfragen

1. Inhaltsverzeichnis

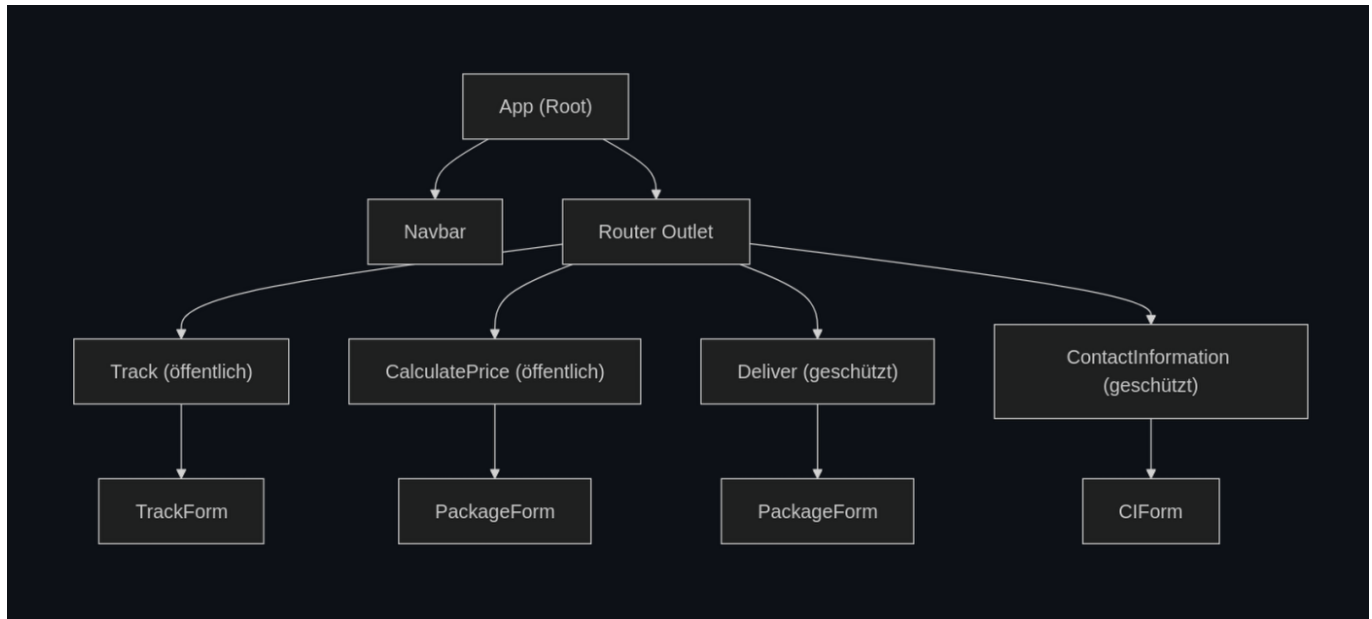
- [2. Dokumentation der Architektur](#)
 - [3. Navigationswege](#)
 - [4. Bebildeter Testlauf](#)
 - [5. KI-Werkzeuge](#)
 - [6. Technische Fragen](#)
 - [6a. URL-Änderungen](#)
 - [6b. Login-Schutz](#)
 - [6c. Dateneingabe-Validierung](#)
 - [6d. Backend-Fehlerbehandlung](#)
-

2. Dokumentation der Architektur

Projekt-Struktur

```
src/
├── app/                                # Angular Komponenten
│   ├── calculate-price/               # Preisberechnung
│   ├── contact-information/           # Kontaktinformationen (geschützt)
│   │   ├── ci-form/                  # Kontaktinformationsformular
│   ├── deliver/                       # Paketaufgabe (geschützt)
│   ├── navbar/                       # Navigation
│   ├── package-form/                 # Paketformular
│   ├── track/                        # Paketverfolgung (öffentlich)
│   │   ├── trackForm/               # Paketverfolgungsformular
│   ├── app.config.ts                 # App-Konfiguration
│   └── app.routes.ts                 # Routing-Konfiguration
├── shared/                            # Geteilte Ressourcen
│   ├── auth/                         # Authentifizierung
│   │   ├── auth.service.ts          # OAuth2/OIDC Service
│   │   ├── auth.guard.ts            # Route Guard
│   │   └── auth.interceptor.ts       # HTTP Interceptor für Token
│   ├── models/                      # TypeScript Interfaces/DTOs
│   │   └── models.ts
│   └── services/                    # Business Logic Services
│       ├── delivery.service.ts       # Delivery-API
│       ├── contact.service.ts        # Kontakt-API
│       └── error-message.service.ts  # Fehler-Management
```

Komponentenbaum



Forms

- **PackageForm**
- **CIForm**
- **TrackForm**

Service-Komponenten Zusammenhang

DeliveryService

Wird verwendet von:

- **Track**: Paketverfolgung und Statushistorie abrufen
- **CalculatePrice**: Preisberechnung durchführen
- **Deliver**: Neue Pakete registrieren

AuthService

Wird verwendet von:

- **Navbar**: Login/Logout-Funktionalität
- **AuthGuard**: Route-Protection
- **AuthInterceptor**: Automatisches Hinzufügen von Bearer-Tokens
- Geschützte Komponenten: Zugriff auf User-Informationen

ErrorMessageService

Zeigt Fehlermeldungen zentral in der Anwendung an.

ContactService

Wird verwendet von:

- **ContactInformation**: Kontaktdaten abrufen und aktualisieren

Technologie-Stack

- **Framework**: Angular 21
- **UI**: Tailwind CSS + DaisyUI
- **OAuth2/OIDC**: angular-oauth2-oidc
- **HTTP Client**: Angular HttpClient
- **State Management**: Angular Signals
- **Forms**: Signal Forms
- **KI-Werkzeuge**: GitHub Copilot, Gemini CLI

Dokumentation mit Compodoc

Ich habe eine Dokumentation mit Compodoc erstellt, da ich das aber erst zum Schluss gemacht habe ist die Dokumentation ziemlich leer.

```
npm run compodoc:build-and-serve
```

Leider gab es keinen Komponentenbaum, deswegen habe ich diesen manuell erstellt.

Ich hätte die Dokumentation mit einem Agenten generieren lassen können, aber ich wollte sicherstellen dass die Dokumentation korrekt ist.

3. Navigationswege

Öffentliche Routes

```
/ (Root)
└─ redirect → /track

/track (Paketverfolgung)
└─ Eingabe: Tracking-ID + PLZ
    └─ Anzeige: Paketdetails + Status-Historie
        └─ (Optional wenn eingeloggt) Anmelden für Benachrichtigungen
        └─ (Optional wenn nicht eingeloggt) Info zur Möglichkeit
Benachrichtigungen zu aktivieren wenn angemeldet

/calculate-price (Preiskalkulation)
└─ Eingabe: Paketdaten (Sender, Empfänger, Abmessungen)
    └─ Anzeige: Berechneter Preis
        └─ (Optional wenn eingeloggt) Möglichkeit Paketdaten für Delivery zu
übernehmen
```

Geschützte Routes (Login erforderlich)

```
/deliver (Paketaufgabe)
└─ Authentifizierung prüfen
    └─ Nicht eingeloggt → Login-Redirect (Keycloak)
    └─ Eingeloggt → Formular anzeigen
        └─ Eingabe: Paketdaten
            └─ Anzeige: Tracking-ID + Bestätigung

/contact-information (Kontaktinformationen)
└─ Authentifizierung prüfen
    └─ Nicht eingeloggt → Login-Redirect
    └─ Eingeloggt → Kontaktdaten anzeigen
        └─ Eingabe: Kontaktdaten hinzufügen
```

Navigation Flow mit Login

```
graph TD
    A[User besucht geschützte Seite] --> B[AuthGuard prüft Login-Status]
    B --> C{ }
    C -- "Eingeloggt → Zugriff gewährt" --> F[Zugriff auf geschützte Seite]
    C -- "Nicht eingeloggt → AuthService.login()" --> D[Redirect zu Keycloak]
    D --> E[User-Login bei Keycloak]
    E --> G[Redirect zurück zur App]
    G --> H[OAuth-Token erhalten]
    H --> F
```

User besucht geschützte Seite
↓
AuthGuard prüft Login-Status
↓
├─ Eingeloggt → Zugriff gewährt
└─ Nicht eingeloggt → AuthService.login()
↓
Redirect zu Keycloak
↓
User-Login bei Keycloak
↓
Redirect zurück zur App
↓
OAuth-Token erhalten
↓
Zugriff auf geschützte Seite

Navbar Navigation

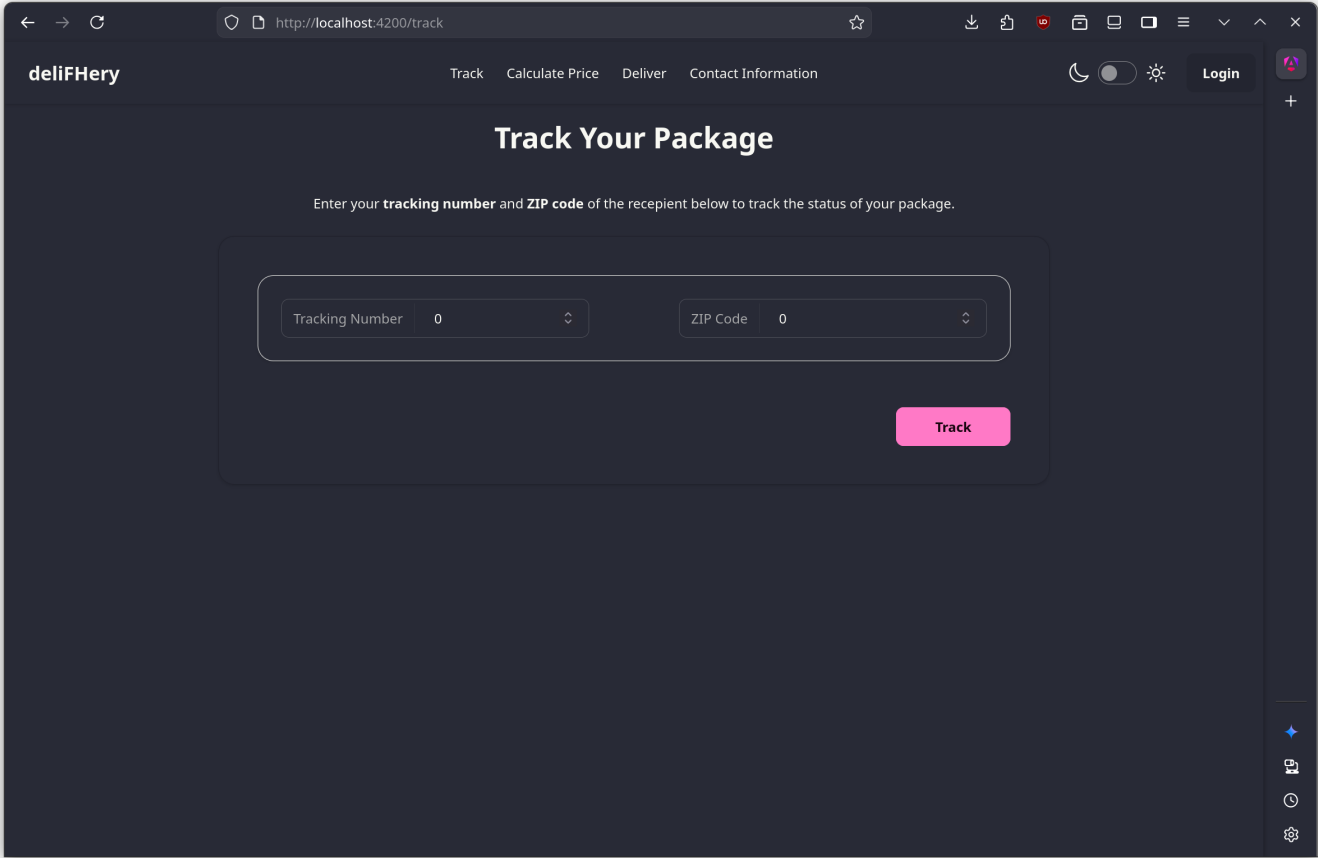
```
graph TD
    A[Navbar (immer sichtbar)] --> B[Track]
    A --> C[Calculate Price]
    A --> D[Deliver (ausgegraut wenn nicht eingeloggt)]
    A --> E[Contact Information (ausgegraut wenn nicht eingeloggt)]
    A --> F[Login/Logout Button]
    F --> G{ }
    G -- "Nicht eingeloggt → 'Login' → Keycloak" --> H[Redirect zu Keycloak]
    G -- "Eingeloggt → 'Logout' → Token löschen" --> I[Token löschen]
```

Navbar (immer sichtbar)
├─ Track
├─ Calculate Price
├─ Deliver (ausgegraut wenn nicht eingeloggt)
├─ Contact Information (ausgegraut wenn nicht eingeloggt)
└─ Login/Logout Button
 ├─ Nicht eingeloggt → "Login" → Keycloak
 └─ Eingeloggt → "Logout" → Token löschen

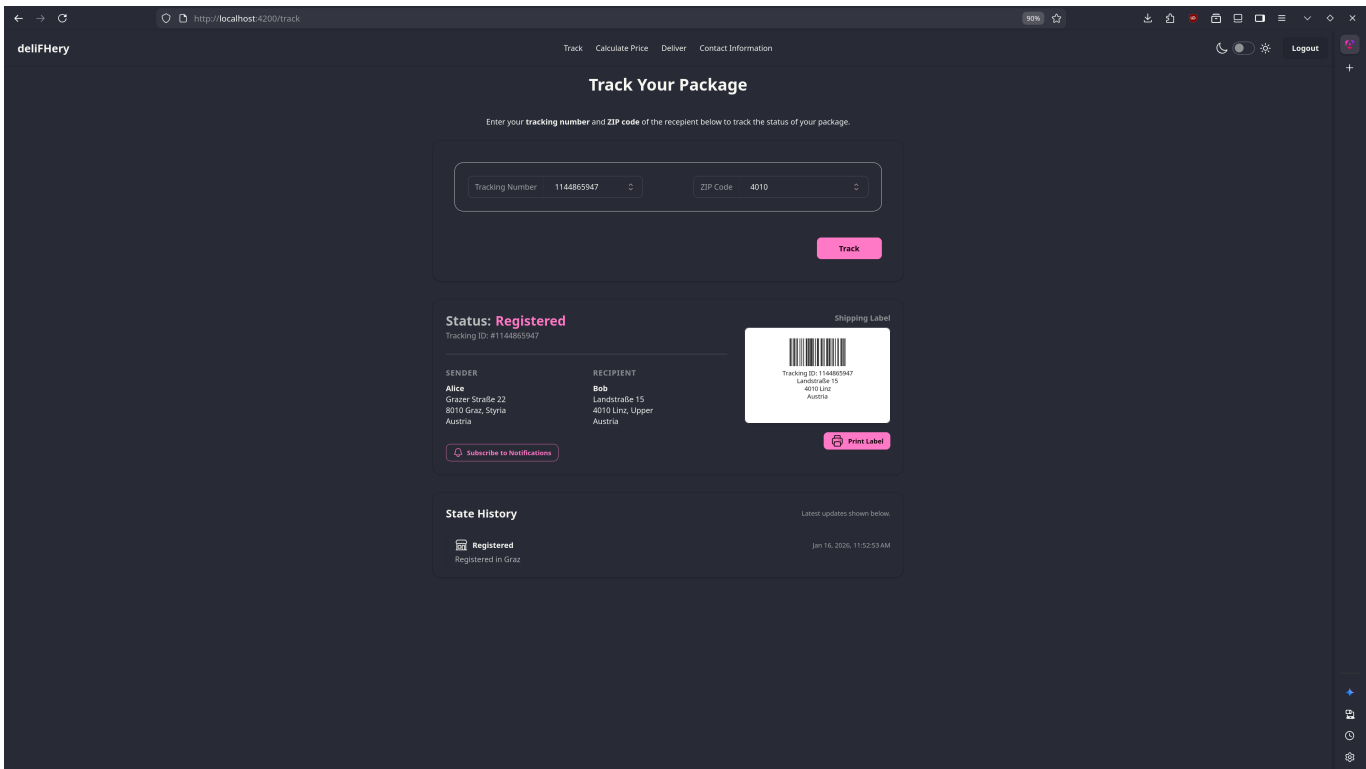
4. Screenshots der Anwendung

Szenario 1: Paketverfolgung (öffentlich)

Track-Seite mit Eingabeformular (Tracking-ID + PLZ)



Screenshot: Angezeigte Paketdetails und Lieferstatus



Szenario 2: Preisberechnung (öffentlich)

Calculate-Price-Seite mit leerem Formular

deliFHery

TrackCalculate PriceDeliverContact Information

Login

Calculate Price

Fill out the package details form below to calculate the shipping price for your package.

Sender

Name*

Sender Name

Country*

State

ZIP Code*

City*

AT

State

0

City

Street*

Street

Recipient

Name*

Recipient Name

Country*

State

ZIP Code*

City*

AT

State

0

City

Street*

Street

Package Details

Weight (kg)*

Width (cm)*

Height (cm)*

Length (cm)*

0

0

0

0

Submit

Ausgefülltes Formular mit Preis

deliFHery

TrackCalculate PriceDeliverContact Information

Logout

Calculate Price

Fill out the package details form below to calculate the shipping price for your package.

Sender

Name*

Alice

Country*

State

ZIP Code*

City*

Austria

Styria

8010

Graz

Street*

Grazer Straße 22

Recipient

Name*

Bob

Country*

State

ZIP Code*

City*

Austria

Upper

4010

Linz

Street*

Landstraße 15

Package Details

Weight (kg)*

Width (cm)*

Height (cm)*

Length (cm)*

5

5

5

5

Submit

Calculated Price: €4,008.75

Deliver this package

Validierungsfehler im Formular

deliFHery

TrackCalculate PriceDeliverContact Information

Login

Calculate Price

Fill out the **package details** form below to calculate the shipping price for your package.

Sender

Name*

dsaf

Country*

dsafkl

State

dsklf

ZIP Code*

4123

City*

dskfl

Street*

lkdsafj

Recipient

Name*

dsklfj

Country*

dslafjsdklafj

State

dsklfaj

ZIP Code*

1241

City*

dsklafj

Street*

dlkfdaj

Package Details

Weight (kg)*

0.5

Width (cm)*

0

Height (cm)*

0

Length (cm)*

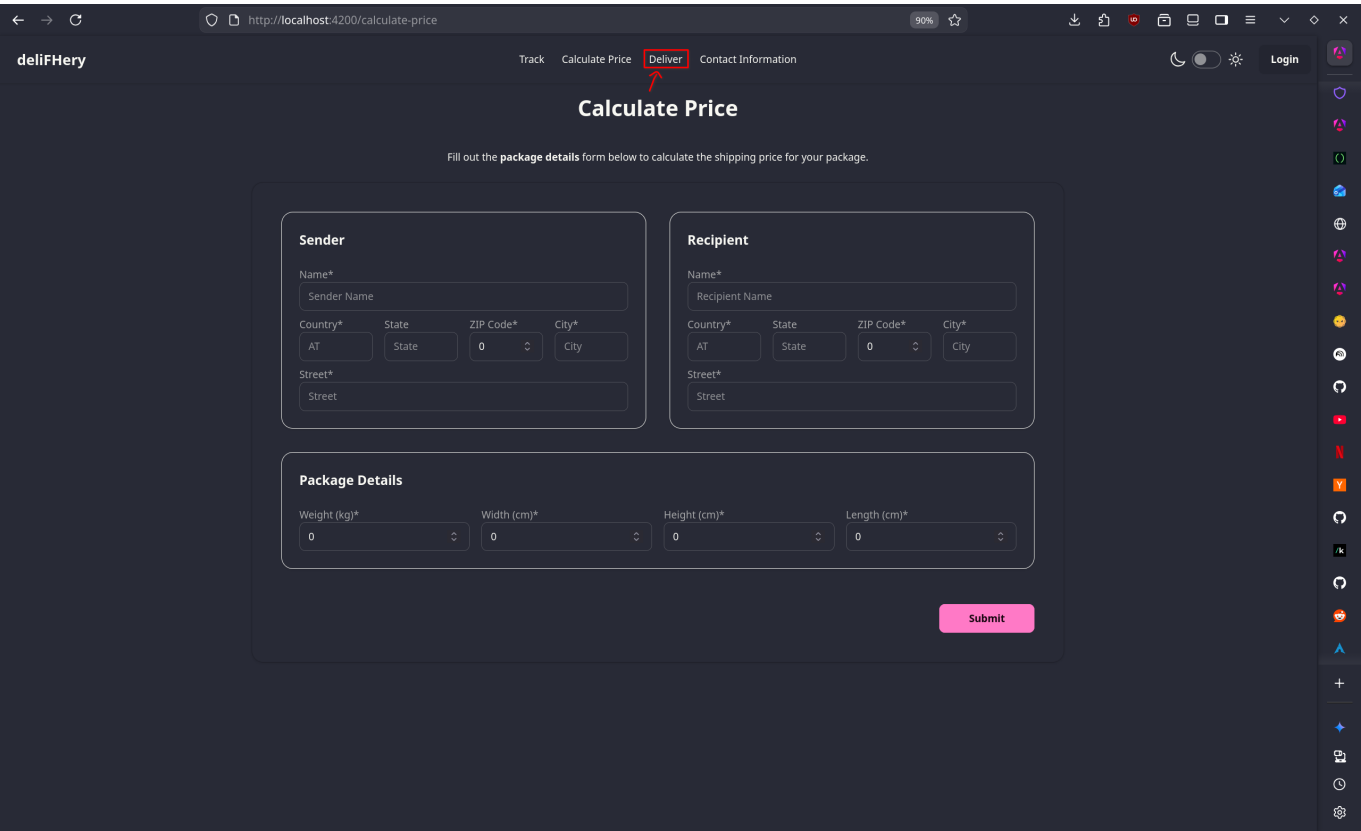
0

Please select a value that is no less than 0.01.

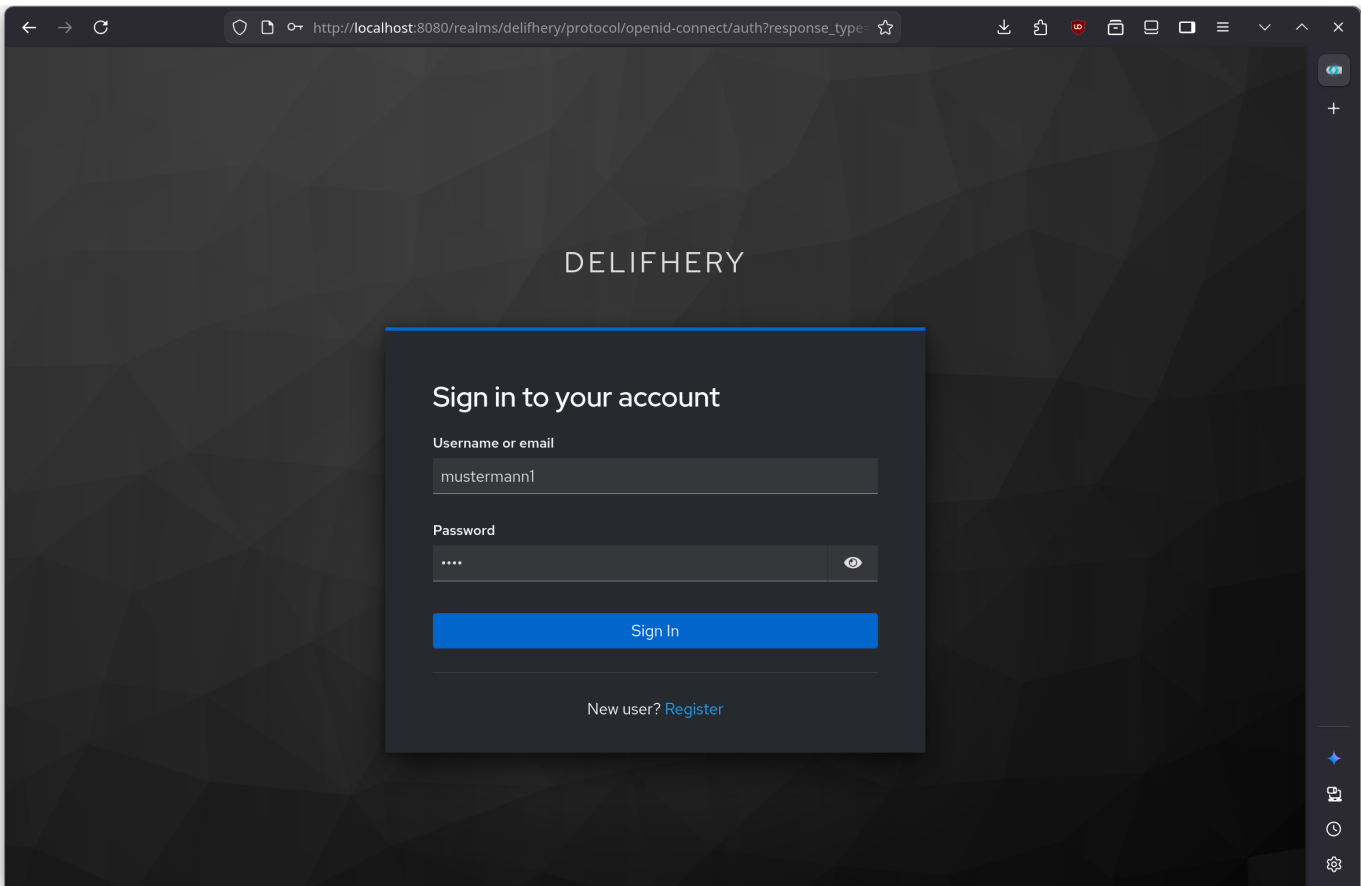
Submit

Szenario 3: Login-Prozess

Versuch, geschützte Seite zu besuchen



Redirect zu Keycloak Login



Account erstellen

The screenshot shows a web browser window with the URL `http://localhost:8080/realms/delifhery/login-actions/registration?client_id=delifhery&`. The page displays a 'Register' form with the following fields and values:

- Username ***: username
- Password ***: (masked with dots)
- Confirm password ***: (masked with dots)
- Email ***: user@mail.com
- First name ***: User
- Last name ***: Name

At the bottom of the form is a blue 'Register' button and a link '< Back to Login'.

Erfolgreicher Login, zurück zur App

The screenshot shows a web browser window with the URL `http://localhost:4200/deliver`. The page displays the 'Register Package' form with the following sections and fields:

- Sender**
 - Name*: Sender Name
 - Country*: AT
 - State: State
 - ZIP Code*: 0
 - City*: City
 - Street*: Street
- Recipient**
 - Name*: Recipient Name
 - Country*: AT
 - State: State
 - ZIP Code*: 0
 - City*: City
 - Street*: Street
- Package Details**
 - Weight (kg)*: 0
 - Width (cm)*: 0
 - Height (cm)*: 0
 - Length (cm)*: 0

A pink 'Submit' button is located at the bottom right of the form.

Szenario 4: Paketaufgabe (geschützt)

Deliver-Seite nach Login

← → ↺

🔒 📄 http://localhost:4200/deliver

90% ☆

📄 📄 📄 📄 📄 📄 📄

🌙 🌞 ⚙️ Logout

deliFHery

Track Calculate Price Deliver Contact Information

Register Package

Fill out the **package details** form below to register your package for delivery.

Sender

Name*

Sender Name

Country*

AT

State

State

ZIP Code*

0

City*

City

Street*

Street

Recipient

Name*

Recipient Name

Country*

AT

State

State

ZIP Code*

0

City*

City

Street*

Street

Package Details

Weight (kg)*

0

Width (cm)*

0

Height (cm)*

0

Length (cm)*

0

Submit

Ausgefülltes Paketformular

← → ↺

🔒 📄 http://localhost:4200/deliver

90% ☆

📄 📄 📄 📄 📄 📄 📄

🌙 🌞 ⚙️ Logout

deliFHery

Track Calculate Price Deliver Contact Information

Register Package

Fill out the **package details** form below to register your package for delivery.

Sender

Name*

Alice

Country*

AT

State

Styria

ZIP Code*

8010

City*

Graz

Street*

Grazer Straße 22

Recipient

Name*

Bob

Country*

AT

State

Upper

ZIP Code*

4010

City*

Linz

Street*

Landstraße 15

Package Details

Weight (kg)*

5

Width (cm)*

5

Height (cm)*

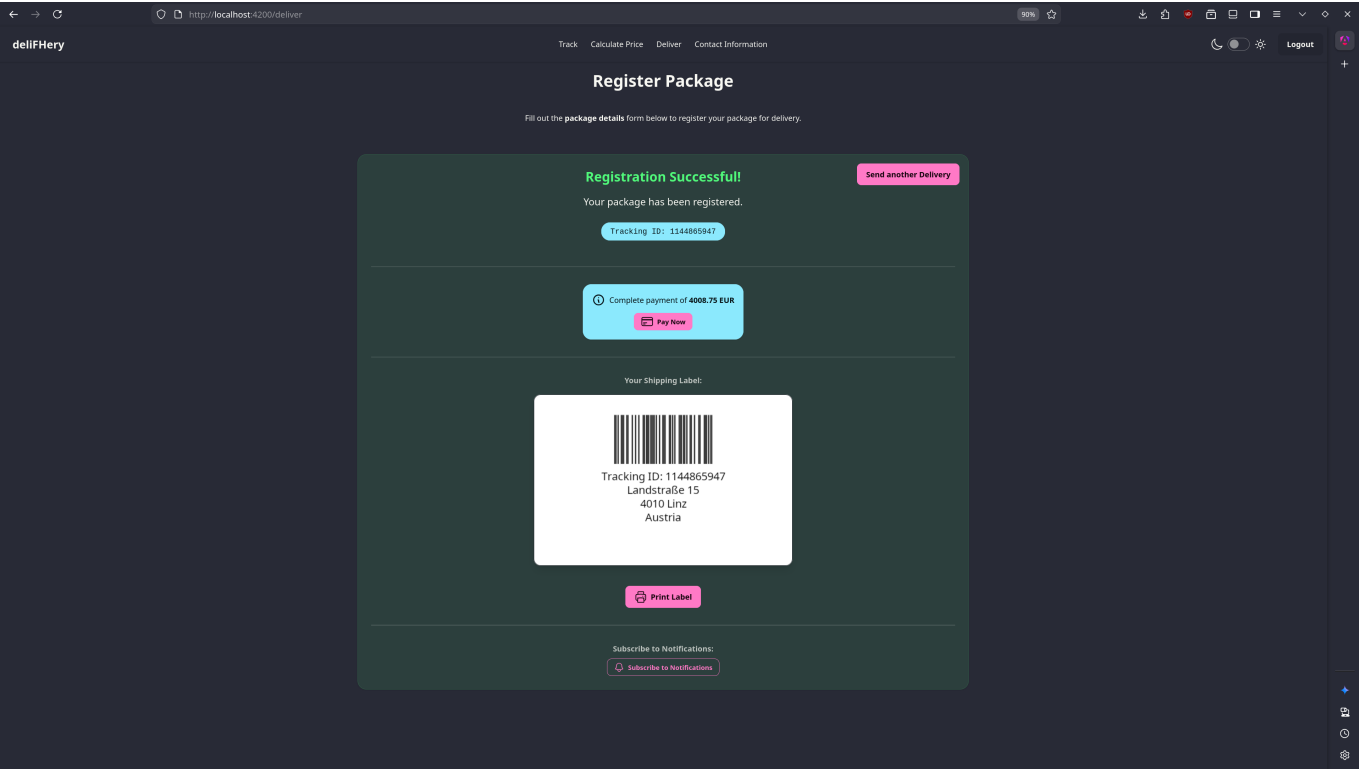
5

Length (cm)*

5

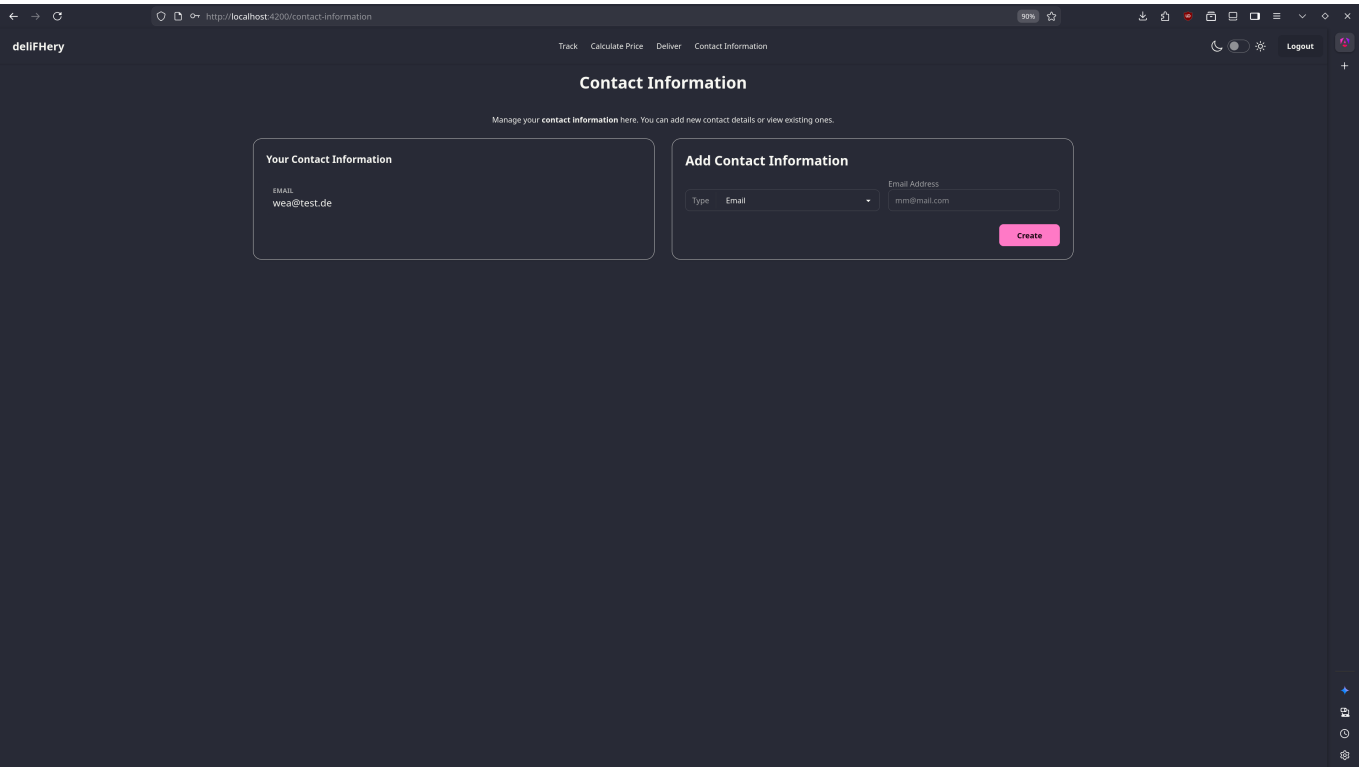
Submit

Erfolgsmeldung mit Tracking-ID

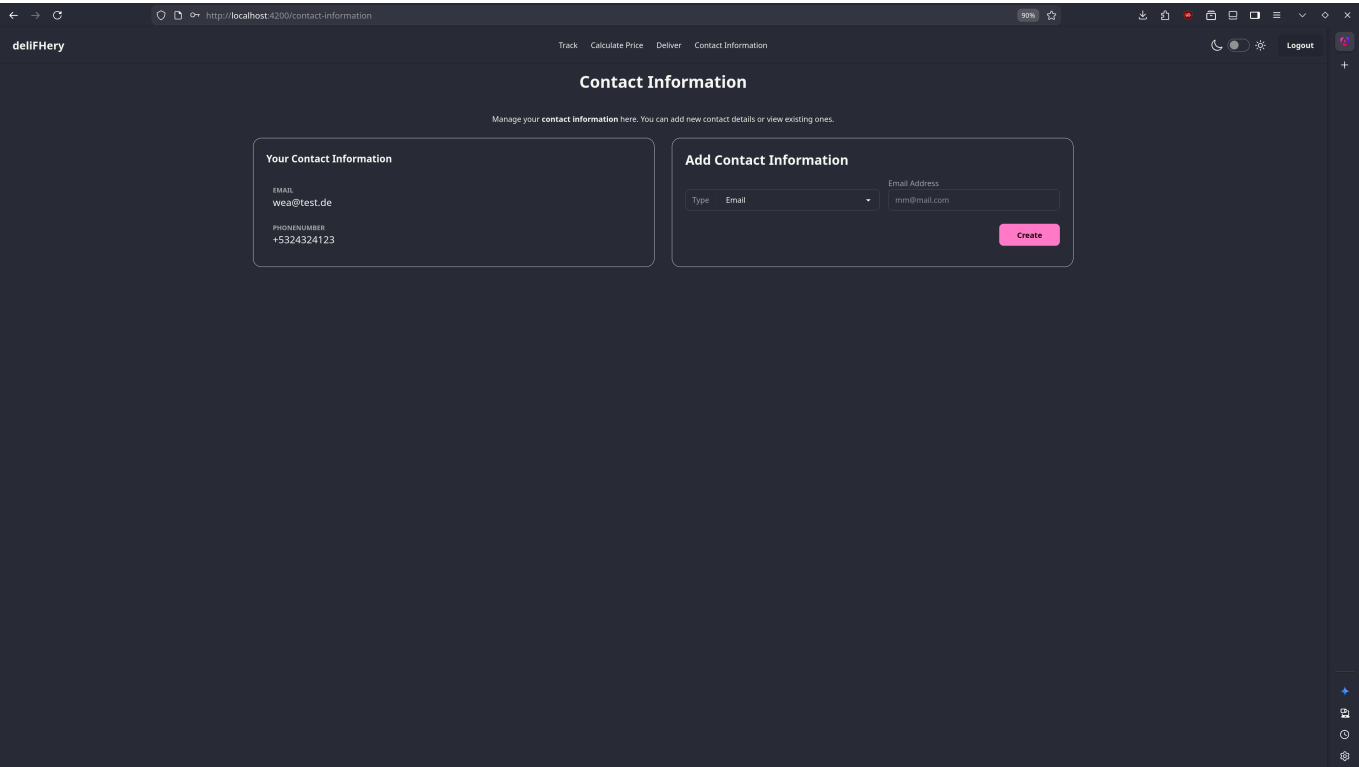


Szenario 5: Kontaktinformationen (geschützt)

Contact-Information-Seite

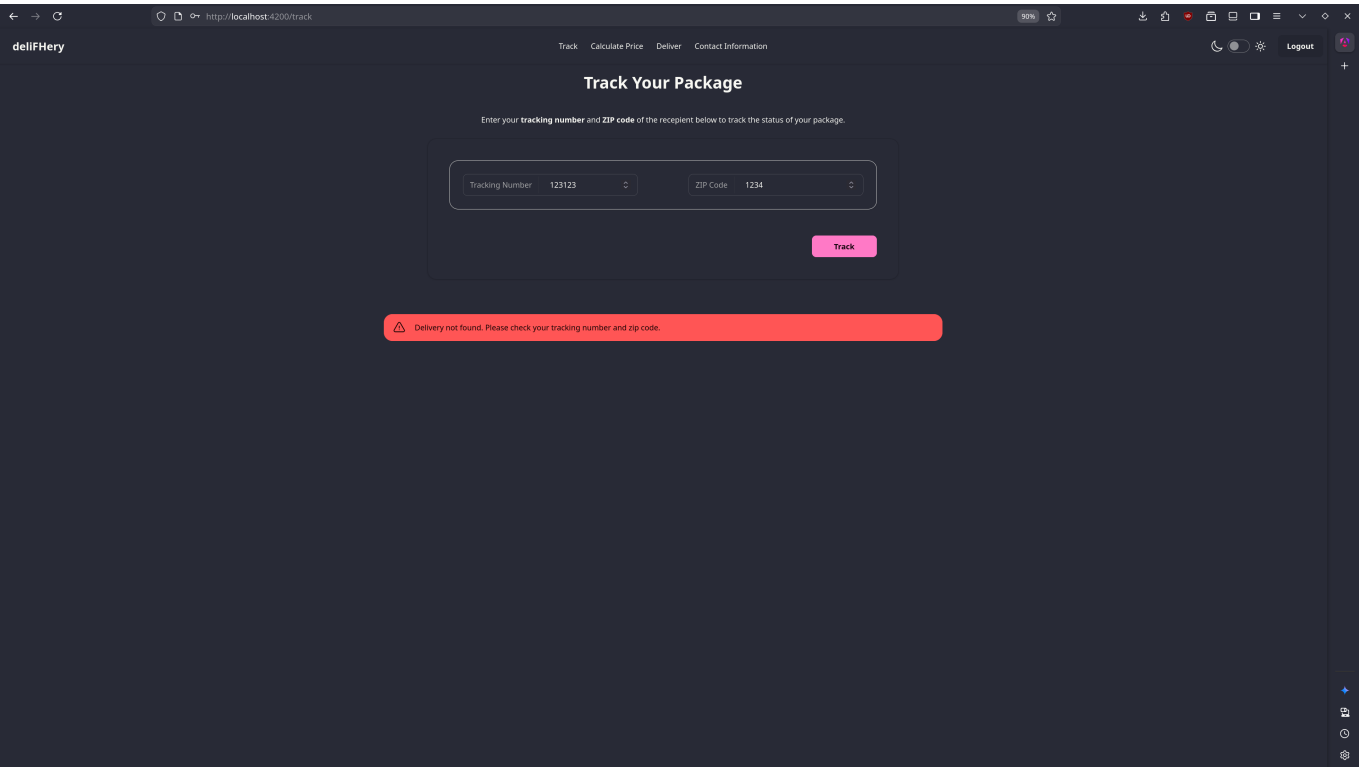


Anzeige/Bearbeitung der Kontaktdaten

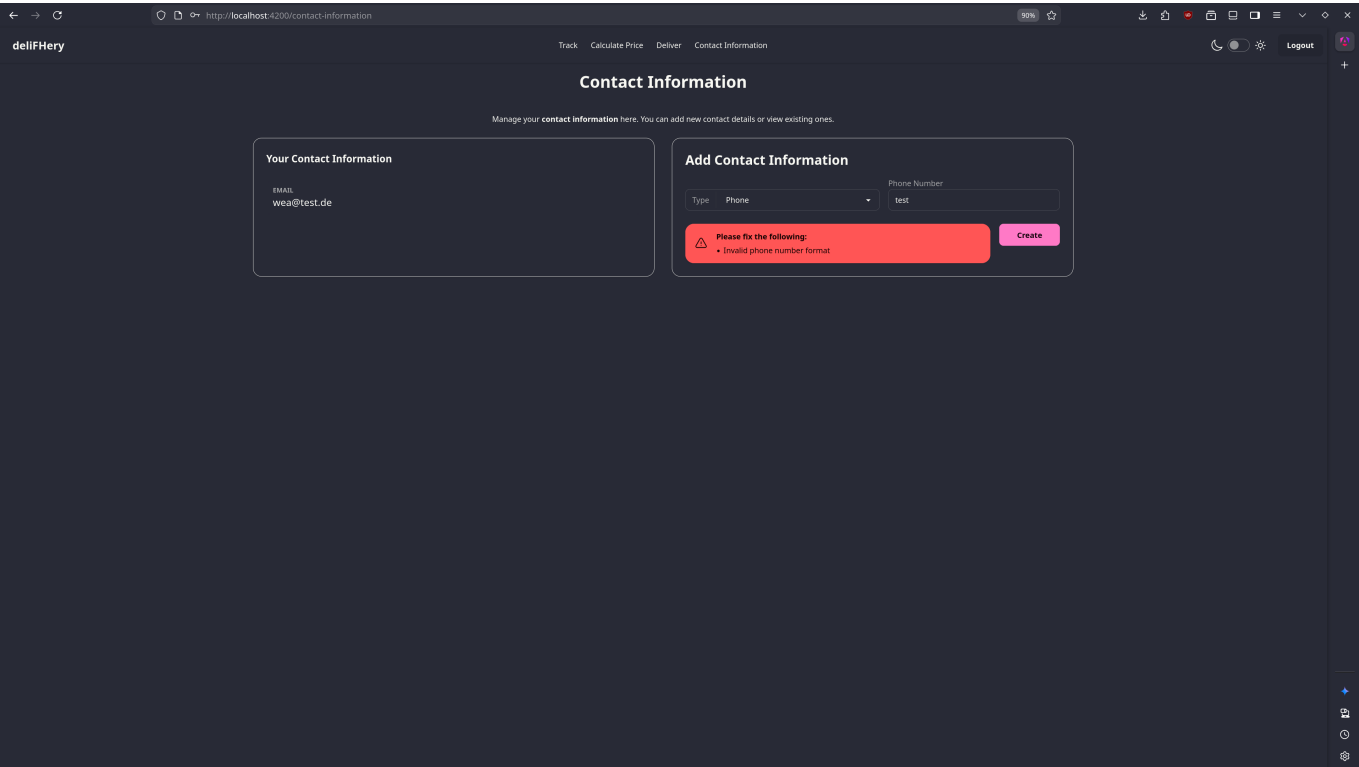


Szenario 6: Fehlerbehandlung

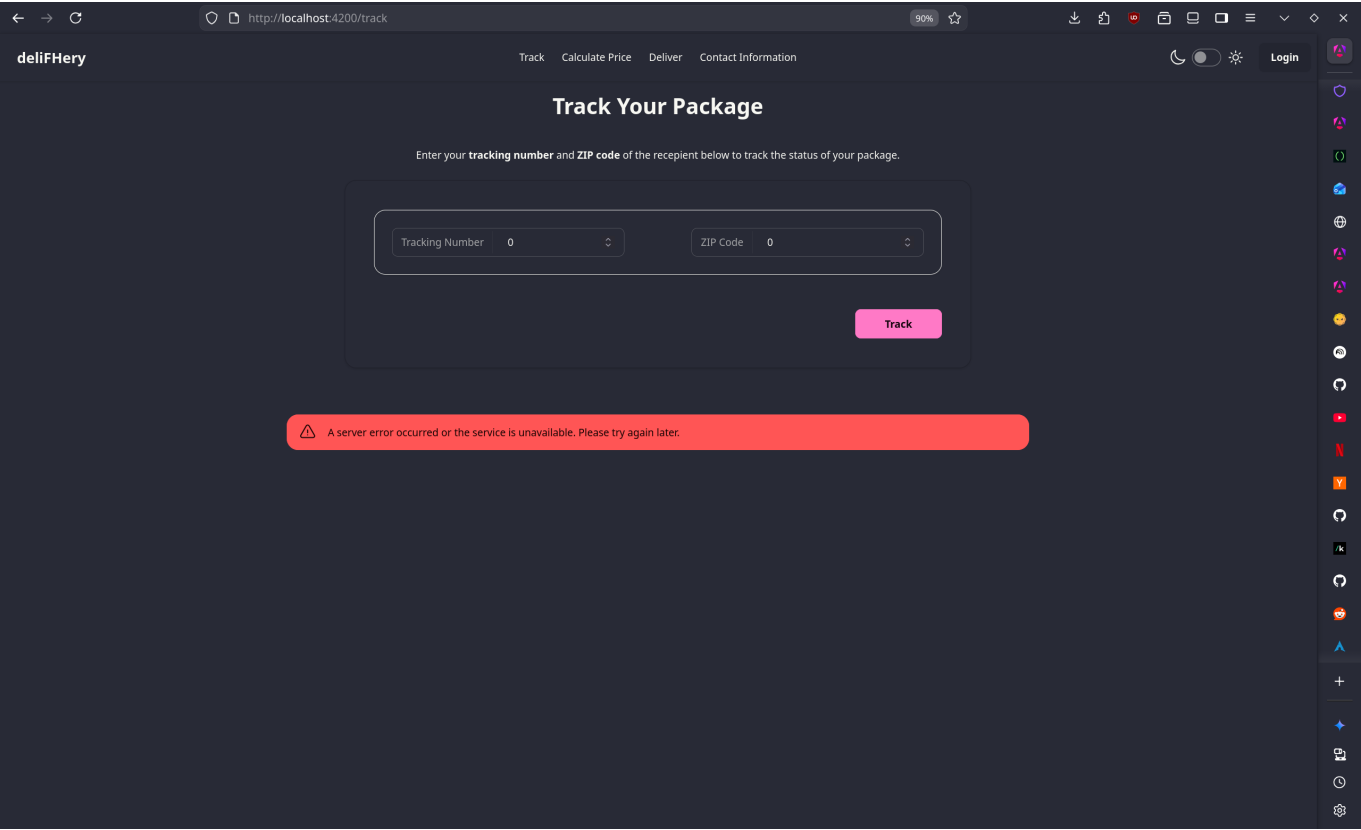
Fehlermeldung bei ungültiger Tracking-ID



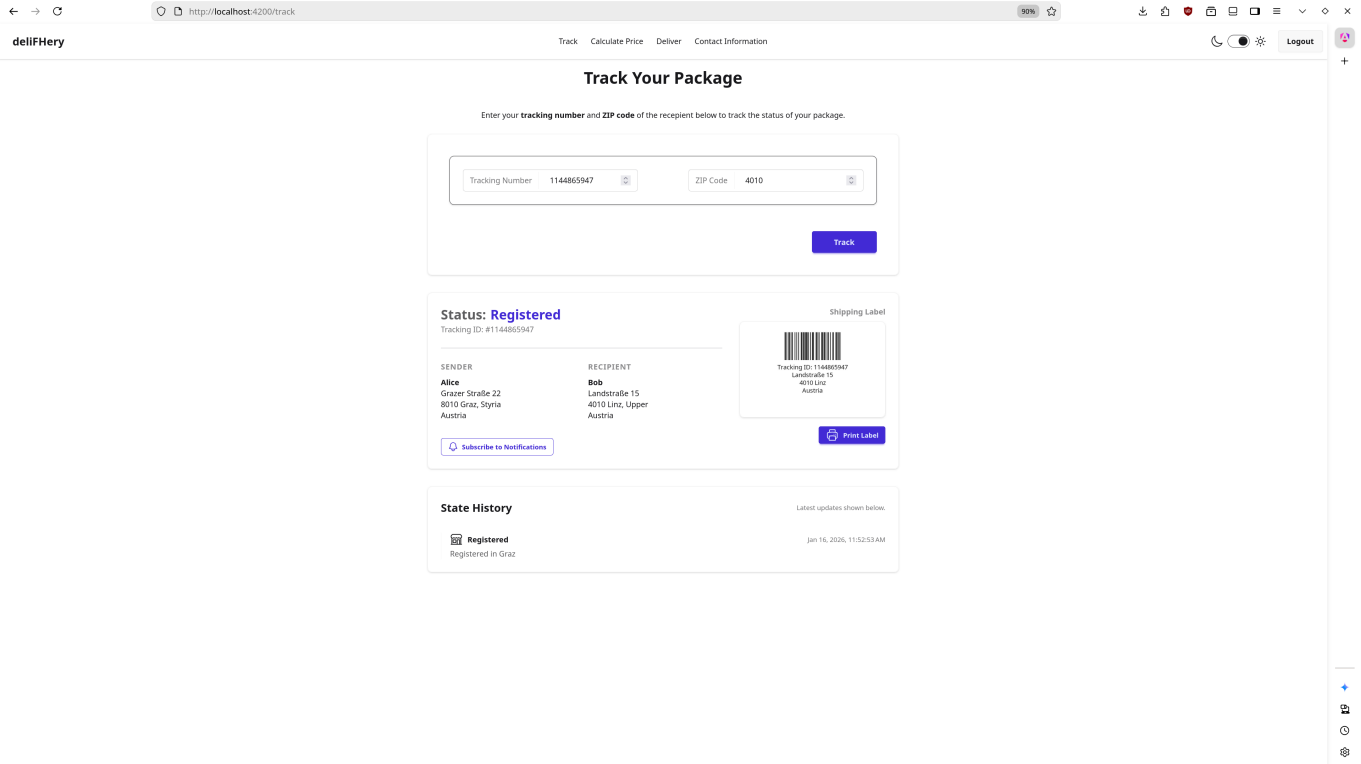
Validierungsfehler im Formular



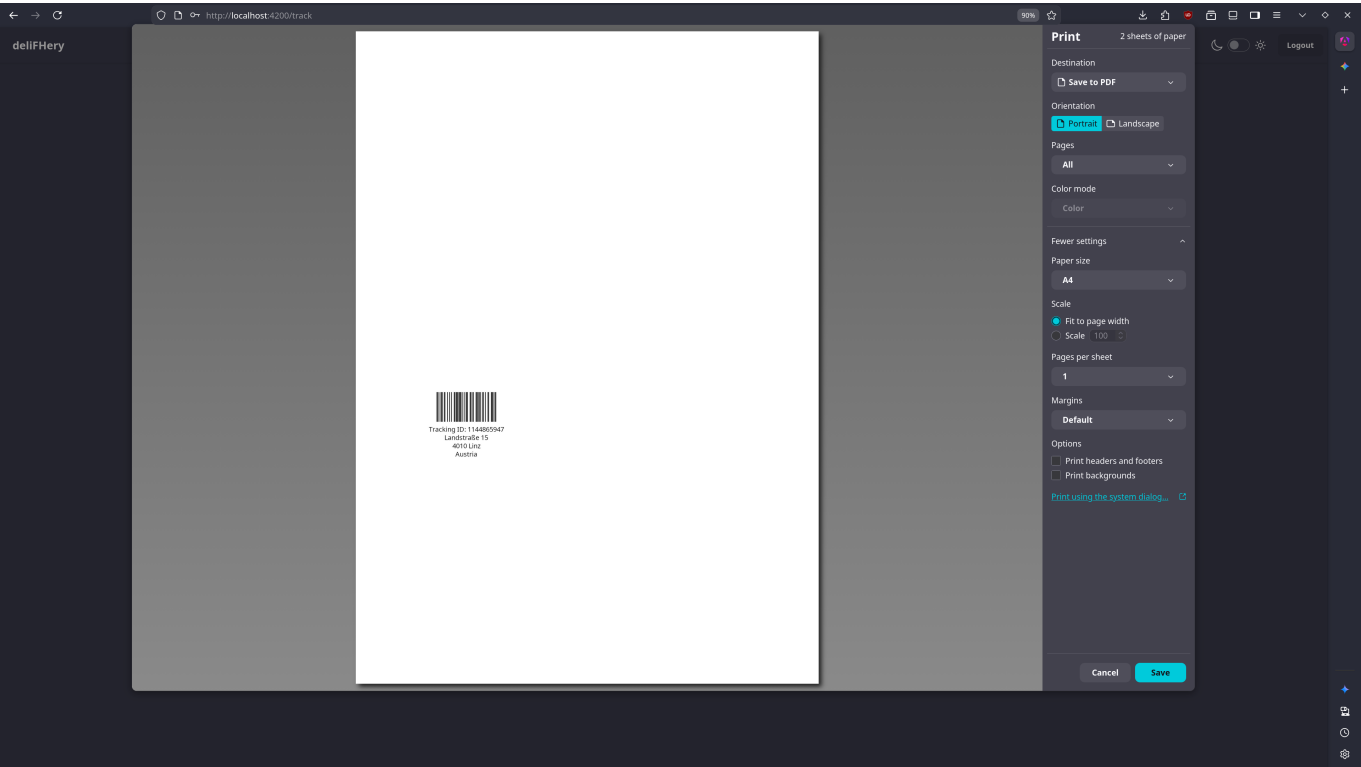
Server-Fehler-Meldung



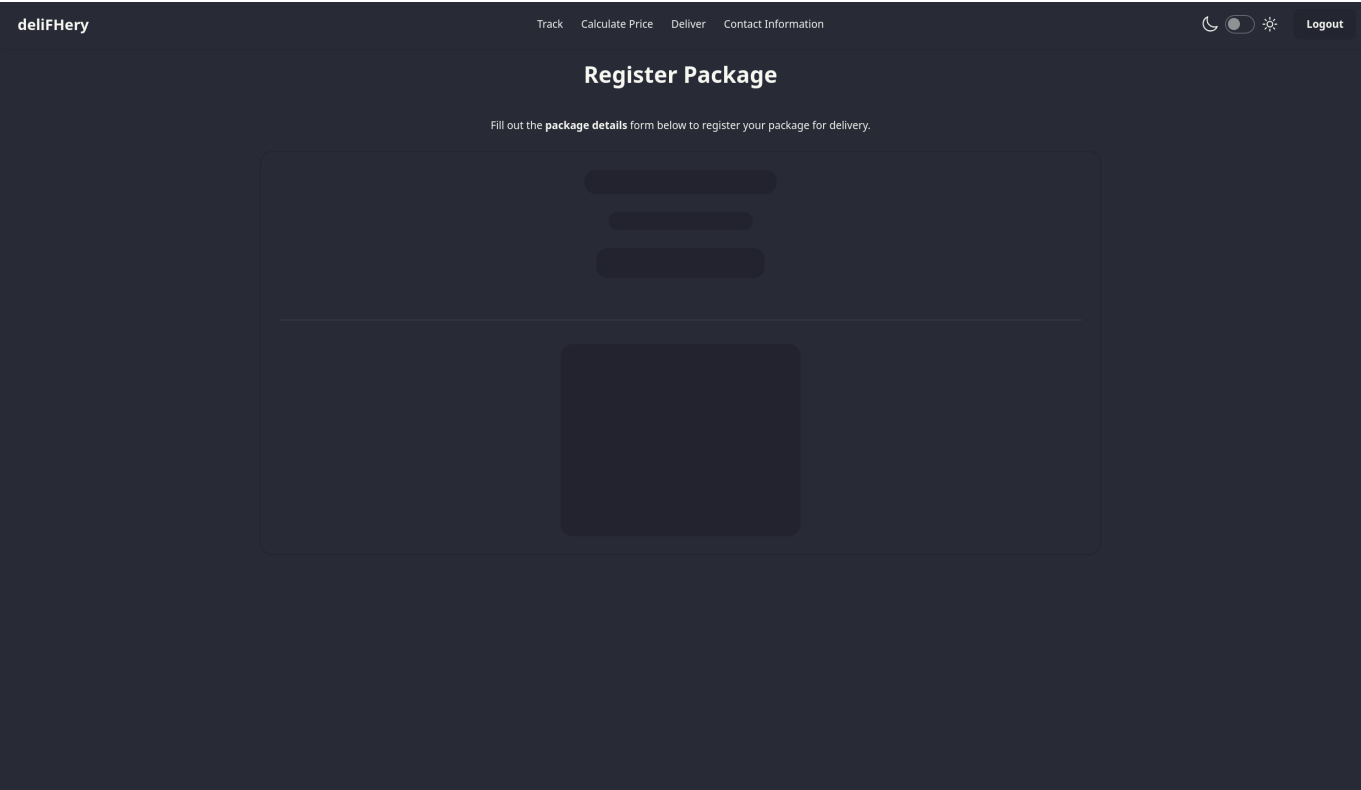
Lightmode



Druckansicht für Label



Skeleton bei Ladezeiten



5. KI-Werkzeuge

Eingesetzte KI-Werkzeuge

Die folgenden KI-Werkzeuge wurden während der Entwicklung eingesetzt:

1. GitHub Copilot

- Auto-Completion in VSCode

2. Gemini / GeminiCLI

- Generierung von unten angeführten Code-Abschnitten

3. Angular MCP Server

- Wie von Angular empfohlen habe ich den MCP Server genutzt

Mit KI erstellte Code-Abschnitte

1. PackageForm Component (`src/app/package-form/package-form.ts`)

- Nach manueller Implementierung von Validierungen mit Signal Forms wurden die restlichen Validierungen generiert.

2. ErrorMessageService (`src/shared/services/error-message.service.ts`)

- Auto-clear Funktionalität nach 5 Sekunden

3. AuthInterceptor (`src/shared/auth/auth.interceptor.ts`)

- HTTP-Interceptor zum Hinzufügen von Bearer-Tokens

4. Print Label Styling (`src/styles.css`)

- CSS für Druckansicht von Labels

5. Styling und Layout

- Tailwind und DaisyUI CSS Klassen-Kombinationen

Schwierigkeiten mit Signal Forms

Auch wenn der Agent Zugang zu dem Angular-MCP Server hatte und in der GEMINI.md File angegeben wurde dass es **Signal Forms** wirklich gibt, wurden zu Beginn von jeder Conversation sehr viele Tokens verschwendet damit Gemini überzeugt sein konnte dass Signal Forms wirklich existieren.

6. Technische Fragen

6a. URL-Änderungen

Was ist zu tun, wenn sich URLs ändern? Wie invasiv ist der Eingriff?

Zentralisierte URL-Verwaltung

Die Backend-URL ist zentral in einer Datei konfiguriert (`environments.ts`), und die Endpunkte selbst in den entsprechenden Services.

Datei: `src/environments/environments.ts`

```
export const environments = {  
  apiUrl: 'http://localhost:5003',  
};
```

Beispiel aus `delivery.service.ts`:

```
import { environments } from '../environments/environments';  
  
export class DeliveryService {  
  private apiUrl = environments.apiUrl;  
  
  calculatePrice(packageData: PackageModel): Observable<PriceModel> {  
    return this.http.post<PriceModel>(`${this.apiUrl}/price`, packageData);  
  }  
}
```

6b. Login-Schutz

Wie stellen Sie sicher, dass bestimmte Seiten nur nach einem Login zugreifbar sind?

OAuth2/OIDC mit Keycloak

- **Library:** `angular-oauth2-oidc`
- **Provider:** Keycloak
- **Flow:** Authorization Code Flow mit PKCE

Implementierung mit AuthGuard

Datei: `src/shared/auth/auth.guard.ts`

```
import { CanActivateFn } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);

  if (authService.isLoggedIn()) {
    return true; // Zugriff erlaubt
  }

  authService.login(); // Redirect zu Keycloak
  return false; // Zugriff verweigert
};
```

Verwendung in Routes

Datei: `src/app/app.routes.ts`

```
export const routes: Routes = [
  { path: 'track', component: Track }, // Öffentlich
  { path: 'calculate-price', component: CalculatePrice }, // Öffentlich
  {
    path: 'deliver',
    component: Deliver,
    canActivate: [authGuard], // Geschützt
  },
  {
    path: 'contact-information',
    component: ContactInformation,
    canActivate: [authGuard], // Geschützt
  },
];
```

AuthService - Login-Status-Prüfung

Ich habe die OAuth2/OIDC-Logik in einem separaten Service gekapselt, da ich zu Beginn glaubte, dass ich noch weitere Funktionen hinzufügen müsste. Ich bin jetzt dabei geblieben da die Funktionsnamen einfacher sind.

Datei: `src/shared/auth/auth.service.ts`

```
export class AuthService {
  private oauthService = inject(OAuthService);

  isLoggedIn(): boolean {
    return this.oauthService.hasValidAccessToken();
  }

  login() {
    this.oauthService.initLoginFlow();
  }

  logout() {
    this.oauthService.logout();
  }
}
```

Bearer-Token fürs Backend

Zusätzlich zur Route-Protection wird jeder HTTP-Request automatisch mit einem Bearer-Token versehen. Das wird benötigt da das Backend ebenfalls den Token validiert.

Datei: `src/shared/auth/auth.interceptor.ts`

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { AuthService } from '../auth.service';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);

  const token = authService.getAccessToken();
  if (token) {
    req = req.clone({
      headers: {
        Authorization: `Bearer ${token}`,
      },
    });
  }

  return next(req);
};
```

6c. Dateneingabe-Validierung

Wie stellen Sie eine korrekte Dateneingabe sicher?

Angular Reactive Forms mit Signals

Die Anwendung nutzt die neuen **Signal Forms** von Angular 21:

Datei: `src/app/package-form/package-form.ts`

```
import { Field, form, max, min, required } from '@angular/forms/signals';

packageForm = form(this.packageModel, (fieldPath) => {
  // Sender validations
  required(fieldPath.sender.name, { message: 'Name is required' });
  required(fieldPath.sender.street, { message: 'Street is required' });
  required(fieldPath.sender.city, { message: 'City is required' });
  required(fieldPath.sender.zipCode, { message: 'ZIP code is required' });
  max(fieldPath.sender.zipCode, 99999, { message: 'ZIP code cannot exceed 5 digits' });
  required(fieldPath.sender.country, { message: 'Country is required' });

  // Recipient validations
  ...

  // Package details validations
  min(fieldPath.weight, 0.01, { message: 'Weight must be greater than 0' });
  ...
});
```

HTML-Binding mit Field-Directive

Datei: `src/app/package-form/package-form.html`

```
<input
  type="text"
  placeholder="Sender Name"
  class="input input-bordered w-full"
  [field]="packageForm.sender.name"
/>
```

Die `[field]` Directive:

- Bindet das Input-Element an das Form-Field
- Zeigt Validierungsfehler automatisch an
- Markiert ungültige Felder visuell

Submit-Validierung

```
onSubmit(event: Event) {  
  event.preventDefault();  
  if (this.packageForm().valid()) { // Validierung wird ausgelöst  
    const credentials = this.packageModel();  
    this.submitPackage.emit(credentials);  
  }  
}
```

HTML5-Validierung

Zusätzlich zu Angular-Validierung nutzen wir native HTML5-Validierung:

```
<input  
  type="number"  
  step="0.01"  
  placeholder="0.0"  
  class="input input-bordered w-full"  
  [field]="packageForm.weight"  
>
```

Visuelle Feedback

Fehlerhafte Felder werden visuell hervorgehoben:

- Fehlermeldungen bei dem Feld bei HTML-Validierung
- Bei Angular-Validierung werden Fehler in der globalen Fehlermeldungsliste angezeigt

TypeScript-Typsicherheit

```
export interface PackageModel {  
  sender: AddressModel;  
  recipient: AddressModel;  
  weight: number;  
  width: number;  
  height: number;  
  length: number;  
}
```

6d. Backend-Fehlerbehandlung

Was passiert, wenn Aufrufe an das Backend Fehler produzieren?

Zentrale Error-Message-Service

Datei: `src/shared/services/error-message.service.ts`

```
@Injectable({
  providedIn: 'root',
})
export class ErrorMessageService {
  private _error = signal<string | null>(null);
  readonly error = this._error.asReadonly();

  showMessage(message: string) {
    this._error.set(message);

    // Auto-clear after 5 seconds
    setTimeout(() => {
      this.clear();
    }, 5000);
  }

  showServerError() {
    this.showMessage(
      'A server error occurred or the service is unavailable. Please try again later.',
    );
  }

  clear() {
    this._error.set(null);
  }
}
```

Fehlerbehandlung in Komponenten

Beispiel aus `src/app/track/track.ts`:

```
onTrack(packageData: TrackingModel) {
  ...
  this.deliveryService
    .trackPackage(packageData)
    .subscribe({
      ...
      error: (err) => {
        if (err.status === 404) {
          this.errorMessageService.showMessage(
            'Delivery not found. Please check your tracking number and zip code.',
          );
        }
        if (err.status === 500) {
```



```
        this.errorMessageService.showServerError();  
    }  
    },  
    });  
}
```

Error Message Display

In der App-Komponente oder Navbar wird die Fehlermeldung angezeigt:

```
@if (errorMessage()) {  
  <div class="alert alert-error">  
    <span>{{ errorMessage() }}</span>  
  </div>  
}
```